Bridge Day 16 INSTRUCTOR KEY

Indexed Mutation and Independent Exam 2 Rehearsal

Learning sequence: Predict - Trace - Explain - Test - Interpret

Monday, July 27, 2026 • 50 points • longitudinal template 07242026_v1

Name		**UIN**		**Section**	

Daily target and independence boundary

Design, mutate, summarize, and test a complete list-processing program from requirements alone.

Allowed tools	All Exam 2 tools: loops, conditions, list reads, append, len, and indexed assignment
Support supplied	Only the requirements, constraints, and test obligations are supplied.
Student decides	The entire algorithm, variable inventory, loop form, mutation order, summaries, and tests.
Scaffold level	Independent construction; no planning questions and no algorithm steps.

Evidence and scoring

Evidence	**Points**	**Secure work shows**
Integrated trace	8	intermediate state, condition results, exact output, and meaning
Diagnosis and repair	6	actual, intended, cause, repair, and a discriminating test
Complete program and tests	36	coherent executable logic, required output, assumptions, and defended tests

Task 1. Integrated trace - 8 points

Produce one complete mutation ledger and exact final output.

```
values = [-2, 7, 14, 3]
changed = 0

for index in range(len(values)):
    original = values[index]
    if original < 0:
        values[index] = 0
    elif original % 2 == 1:
        values[index] = original * 2
    else:
        continue
    changed = changed + 1

print(values, changed)
```

Your task

1. For every index, record original value, selected branch, final value, list state, and changed; then give exact output.

Answer 1. Index 0: original -2, negative branch, final 0, list [0, 7, 14, 3], changed 1. Index 1: original 7, odd branch, final 14, list [0, 14, 14, 3], changed 2. Index 2: original 14, else/continue, list unchanged, changed remains 2. Index 3: original 3, odd branch, final 6, list [0, 14, 14, 6], changed 3. Exact output: [0, 14, 14, 6] 3.

Task 2. Diagnose and repair - 6 points

The intent is to replace every negative value, including the last item, and count each replacement.

```
values = [4, -2, 8, -5]
changed = 0

for index in range(len(values) - 1):
    if values[index] < 0:
        values[index] = 0
        changed = changed + 1

print(values, changed)
```

Your task

1. Give actual and intended output, identify the boundary defect, repair it, and name a test that would fail if the last index were skipped again.

Answer 1. Actual output is [4, 0, 8, -5] 1. Intended output is [4, 0, 8, 0] 2. range(len(values) - 1) stops before the last valid index; repair with range(len(values)). Protective test [-1] must become [0] with changed 1.

Task 3. Construct a complete program - 36 points

Program specification

- Use temperatures = [18, -4, 31, 22, 50].
- Mutate every value below 0 to 0 and every value above 40 to 40.
- Count how many positions change.
- Using the final mutated values, compute total, average, and the count from 20 through 40 inclusive.
- Print the final list, changed count, total, average, and in_range count in that order on separate lines.
- Do not use sum, min, max, comprehensions, slicing, or helper functions. Provide two tests with expected results, including a no-change case and a first/last-change case.

Program workspace

Model program

```
temperatures = [18, -4, 31, 22, 50]
changed = 0
total = 0
in_range = 0

for index in range(len(temperatures)):
    if temperatures[index] < 0:
        temperatures[index] = 0
        changed = changed + 1
    elif temperatures[index] > 40:
        temperatures[index] = 40
        changed = changed + 1

    total = total + temperatures[index]
    if 20 <= temperatures[index] <= 40:
        in_range = in_range + 1

average = total / len(temperatures)
print(temperatures)
print(changed)
print(total)
print(average)
print(in_range)
```

Reasoning: Each position is mutated before it contributes to total or in_range, so every summary describes final state rather than original state.

Reasoning: An index loop is required because assignment must write back to a list position. The average is safe because the supplied list is nonempty.

Task 3 continued. Test and defend - included in 36 points

Continue code if needed.

Expected tests and results

1. Supplied list -> [18, 0, 31, 22, 40], changed 2, total 111, average 22.2, in_range 3.
2. No-change [20, 30] -> [20, 30], changed 0, total 50, average 25.0, in_range 2.
3. First/last change [-1, 20, 45] -> [0, 20, 40], changed 2, total 60, average 20.0, in_range 2.

Required tests, expected results, and brief defense

Scoring guidance: 6 correct traversal and writable indexes; 8 complete boundary mutation; 5 changed count; 6 final-state total/average; 4 inclusive range count; 3 exact output; 4 tests and justification.

Bridge Day 16 INSTRUCTOR KEY

VIP Evidence Handoff

Learning sequence: Predict - Trace - Explain - Test - Interpret

Contract 07a04826c975 • longitudinal template 07242026_v1

Why engineers use this page

Rehearse indexed traversal, filtering, compound Boolean evidence, and the Day 6 Independent Program Studio inside the announced loop/list Exam 2 boundary; do not introduce function structure today.

Decision boundary: Code is useful only when its stored state, visible result, assumptions, units, limits, and tests support a defensible engineering interpretation.

Construct readiness before independent work

New authoring tools: string traversal, list traversal, list index read, list index write, string index read, slice expression, membership test, parallel list indexing, compound filter
Carried authoring tools: assignment, print call, arithmetic expression, input call, int conversion, float conversion, f string, comparison expression, boolean operator, if elif else, for loop, range call, append call, len call
Supplied-code exposure only: none
An exposure-only construct may be traced in complete supplied code, but the independent task will not require students to invent it before its authoring day.

Notebook cells and task constructs

Activity	Work	Stable cells	Required authoring constructs
18.25	Reverse Values	`18-25-work / 18-25-transfer / 18-25-cold`	append call, for loop, range call
18.26	Values At Or Above Cutoff	`18-26-work / 18-26-transfer / 18-26-cold`	append call, comparison expression, for loop
18.27	Feasible Workstations	`18-27-work / 18-27-transfer / 18-27-cold`	append call, boolean operator, f string, float conversion, for loop, input call, int conversion, range call

Readiness evidence

1. For Activity 18.25, write the exact index sequence used to visit every value in reverse order without changing the original list.

   Model response: Start at len(values) - 1, stop after index 0, and step by -1. The original list is read only; each visited value is appended to a separate result list.

2. For Activities 18.26 and 18.27, state the keep condition and name one boundary case that must be tested before Exam 2.

   Model response: 18.26 keeps values at or above the inclusive cutoff. 18.27 keeps a workstation name only when the same index satisfies both inclusive constraints. Strong tests include equality at each boundary and a case that satisfies only one constraint.

Timing and help trigger

Record a first prediction before running. If you still lack an executable plan after about 8 focused minutes, use the linked ThinkCSPy refresher or the supplied model. If two attempts fail for the same named requirement, write the exact mismatch and ask a peer mentor or instructor a targeted question. Timing is diagnostic evidence, not a speed grade. **Start or elapsed minutes:** ____________________ **My help trigger:** ____

Engineering Evidence Record

Complete one record from a model, transfer, or Independent Program Studio build. Generic statements are not sufficient; use actual values, a real revision, and one concrete next test.

Evidence field	Record
Prediction	A specific expected state or output before execution.
Observed Result	The actual state or output, including a value or exact text.
Revision Reason	The defect or mismatch and the change that addresses it.
Engineering Meaning	How the evidence changes or supports the engineering decision.
Units Or Not Applicable	The physical unit, or the evidence type when no unit applies.
Assumption	One condition accepted as true for this model or rule.
Model Limit	One situation the result does not justify or predict.
Next Test	A concrete boundary, invalid, or alternate case with an expected result.
Help Trigger	The exact unresolved point that would trigger a targeted question.
Minutes Used	Approximate minutes from first prediction through revision.

Notebook and submission procedure

1. Attempt, predict, run, inspect, and revise before opening a reveal.
2. Complete the end-of-notebook Independent Program Studio without a reveal or copied solution. Only those visibly labeled Studio cells are scored for independent mastery.
3. Save or download the completed notebook as one `.ipynb` file.
4. Upload that one notebook to the matching Gradescope assignment. Do not export a `.py` file or create a ZIP.

Generated from the canonical VIP construct and evidence contract; workbook planning and executable notebook work remain distinct.